Cloud Computing essentials

Page No.:
Date: 13/1/25

* Types of cloud services (Models) :-
  → IaaS → Infrastructure as a Service
  → PaaS → Platform as a services
  → SaaS → Software as a Service

* IaaS → is means of delivering computing infrastructure as on-demand services.
  ex :- AWS, Azure, Google Cloud, IBM Cloud., EC2
  → OS level responsibility Azure VM
* PaaS is cloud delivery model for applications composed of services managed by 3rd party.
  ex :- AWS Lambda, Google app engine, Azure Webapps
  → code level responsibility → AWS elastic beanstalk
* SaaS allows users to run existing online applications and it is a model software that is deployed as a hosting service and it is accessed over Output Refreshed / Written text the internet.
  - It is used for the development or deployment of modern applications.
    ex - Salesforce, Zoom, Microsoft 365, Google workspace apps
    Adobe Creative cloud.
  → Consumer → Youtube, Vimeo, Twitter, Salesforce
* NaaS

- AWS Services
- Hypervisors → VMWare
- Bare Metal Hypervisors

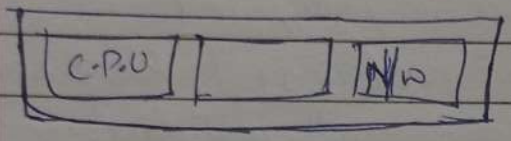
Date: 15/1/25

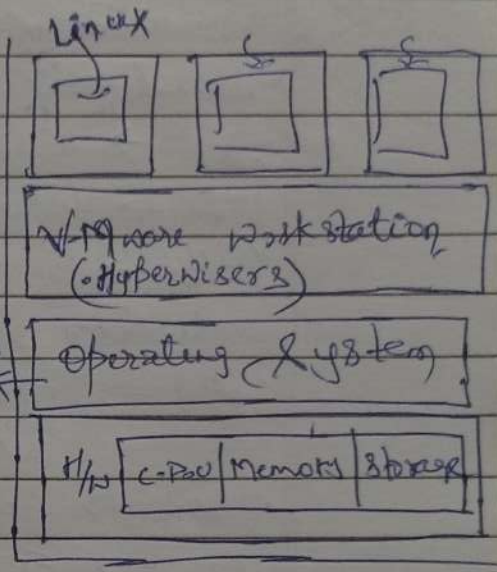
Cloud Computing

- Evolution / History of cloud
- Benefits of cloud.
- Characteristics of cloud.

Cloud

- VM → Virtual Machine





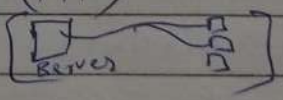
without O.S → Bare Metal Hypervisors

- download VMware Workstation
- ISO file of Ubuntu

Simulation → virtual
Emulation → H/W to S/W

16/1/25 # History of Cloud computing (1970)

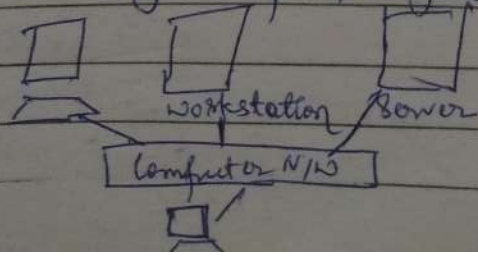
- Client-Server Architecture



drawback:

- Scalability
- Resource sharing
- Cost

- Distributed System:
  - Loosly coupled System
    - parallel working
    - Resources can be shared over N/W.



Cloud computing



Events

1961 → John McCarthy
We can share resources over N/w as utilities.

1969 → ARPANET [internet base]

1970 → Virtualization software [VM ware]

1997 → In 1997 Emori university, professor ram nath chenappa coined the term cloud computing.

1991 → Salesforce company

2002 → AWS by amazon

2006 → EC2

2009 → Google play by application software.

2009 → also Microsoft Azure came.

* Def of cloud computing according to NIST (national institute of standard & technology)

→ 'Cloud computing is the model for enabling ubiquitous (mobility), convenient, on demand network access to a shared pool of configurable computing resources.

eg:- N/ws, servers, storage, applics and services that can be rapidly provisioned and released with minimal management effort or service provider interaction.

Business → capex (Headcount, working capital)
→ opex (operational exp_ture)

Page No.:
Date: / /

Characteristics of cloud computing

1) On-demand ~~self~~ service → (Automation)
2) Broad N/w access
3) Resource Pooling (Big Players) (By data centres)
4) Rapid Elasticity (cost-use billing) scalability ↑↓ req.
5) Measured service (cost upon use)
6) Multi Tenancy (shared resources to many by VM's) ((~~store resources~~))

Benefits and need / Adv & disadv of cloud.

- ~~Scalability~~ scalability
- High availability & business continuity (24x7)
  (if high demand, server swap by hot swap)
- cost efficiency (pay as you go)
- Fast deployment & market release.
- Remote access
- Collaboration and productivity.
- Elasticity (like cricket streaming), pay as u go, on demand, shared resource for everyone

Disadvantages

- Limited control or access over the underlined hardware.
- Security of Data.
- Vendor locker is problem that may arise.
  (you have to use company services too)
- Complexity in integration of existing system with cloud.
- Unexpected expenses can be there while migrating to cloud.

Page No.:
Date: 17/1/25

Cloud Deployments Model

→ Private cloud → my business my rule. (own hardware, install your software)
→ Public " → economy of scale, Elasticity
→ Hybrid "
→ Community cloud (By group of people/company for research, for govt, specific goal)

** Private Cloud (ownership is private) (access allowed by them)
ex: Helbet Packert (HPE), Toshiba, IBM (any hardware, security, compliance)

** Public cloud (resources are rented to users)
ex: Google cloud, AWS, Azure

* Hybrid cloud → mixture of private & public.
(by companies) → Internet or Private links

& Multi cloud → AWS, Azure, Open Stack

Service Models

* ~~IaaS~~, ~~PaaS~~, ~~SaaS~~

→ Compute Power (VM)
→ Storage
→ N/W

eg: EC2, S3 (storage), VPC, Route 53, Google Drive
↓
Instances/VMs

* PaaS

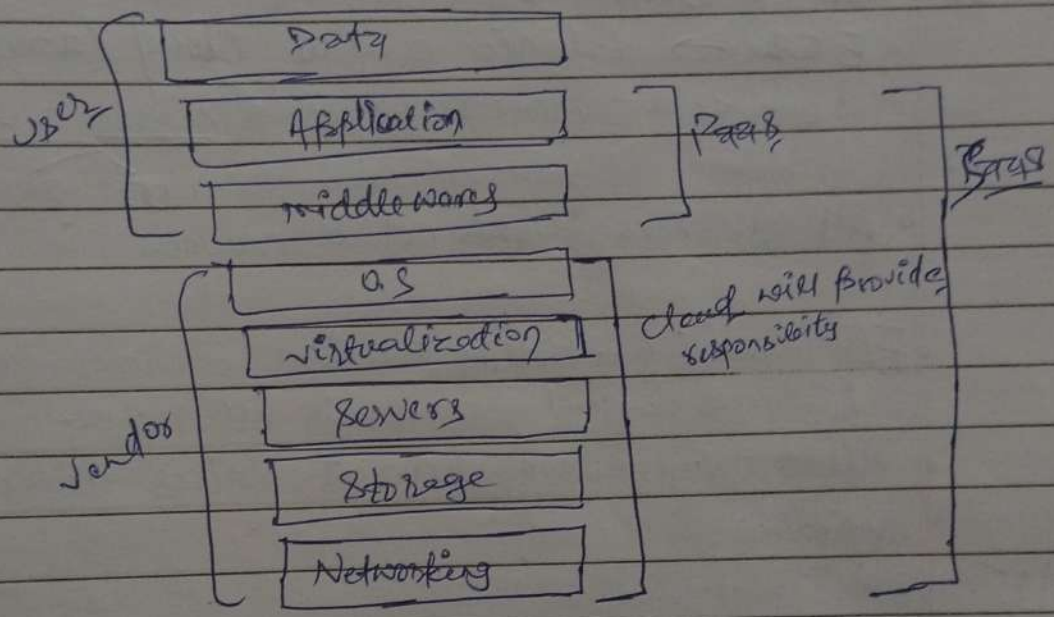
→ Infrastructure
→ Hardware, OS

ex: Elastic Beanstalk (Web Servers)

*SaaS

→ hardware, platform, application

ex:- G doc, form, gmail



Shared responsibility model

20/1/25 Practical

- Lab 1
  - Creating virtual Machine

prior req.
- VMware player, virtual box
- Hyper V
- Linux distros

* Creation of VM

Step 1 → to enable the VTX (virtualization in windows)

- Enter the BIOS → adv. setting → virtualization → enabled/disable
- We need a hypervisor to create the VM
- VMware player, workstation, virtual box (oracle)
- Hyper V → VM
- VMware Player → VM

Page No.:
Date : / /

→ Install virtual machine (Oracle VirtualBox, VMware)
→ Install ISO file (linux dist)
→ Create a virtual machine.

~~EC2~~

AWS services

- EC2 → Elastic compute cloud (IaaS service)
- I can take servers on rent (server instance)
  - GUI
  - shell/SDK
- All services might not be available in all regions like IAM
- IAM service, Billing & costing services are available in all regions (identity access management)
- All services have different prices according to regions.

20/1/25

→ Virtualization is the enabling technology behind the cloud computing
→ Virtualization refers to creating virtual resources like servers, storage, networking, operating system etc.
→ The main goal of virtualization is to manage workloads by radically transforming traditional computing to make it more scalable.
→ The most common form of virtualization is the operating system level virtualization
In O.S level virtualization → it is possible to run multiple O.S on a single piece of hardware.

Page No.:
Date: / /

⇒ Virtualization technology involves operating the physical hardware by emulating the hardware resources using software.

→ A VM is nothing but a data file on a physical computer that can be moved & copied to another computer just like a normal data file.

→ Characteristics of virtual machines

* Virtualization has 3 characteristics that makes it ideal for cloud computing

i) Partitioning → In virtualization many applications and operating systems are supported in a single physical system by partitioning the available resources.

ii) Isolation
Each virtual machine is isolated from the physical system and other virtual machines that are hosted on the same physical system. In case if one virtual machine crashes then it does not effects another virtual machine.

iii) Encapsulation
A virtual machine can be represented as a single file so that it can be easily identified based on the service it provide. It can protect an appl^n who interfare with another appl^n.

Page No.:
Date: / /

* Using Hypervisors to create Virtual Machines

→ Hypervisors is the software that acts as a abstraction layer b/w the physical machine and the virtual machine.

→ Hypervisors are used to create the VMs.

→ With the help of hypervisors it is possible to partition the physical computer resources.

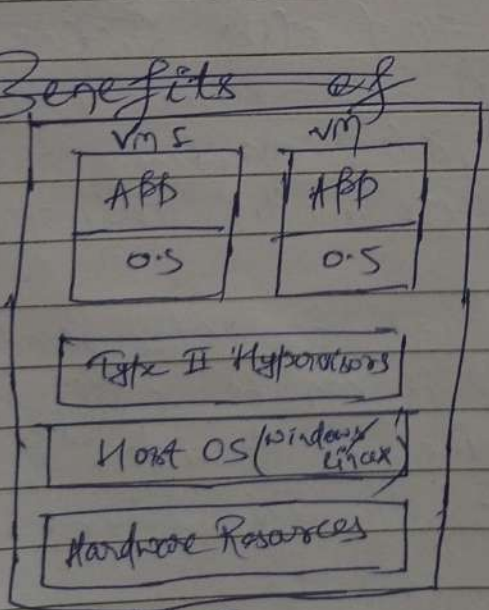
* Types of Hypervisors

i) Native Hypervisors / Bare-Metal Hypervisors / Type one :

This hypervisor sits directly on the hardware platform and thus provide the better performance.

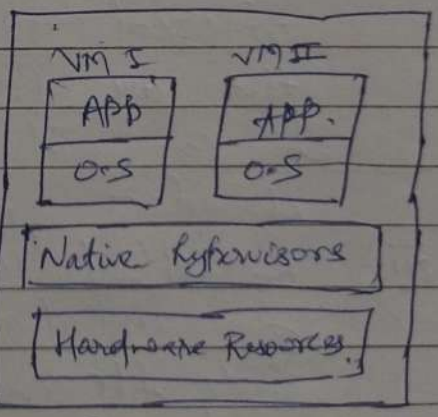
ii) Hosted Hypervisors / Type 2 Hypervisors.

Hosted : runs on the host O.S.



Type II

eg:- Virtual box ,
VMware.



Type I

ex:- Hyper-V (Windows)
- KVM (Linux)

* Benefits of Virtualisation :-

→ We can run multiple OS environment in a single machine.
→ This type of organization is very good for testing and debugging.
→ Easy for portation.
→ It is easy to clone digital machine.
→ Isolation is one of the major benefits in which one app2 running on one VM does not interfares with app2 running on same machine.
→ If one VM crashes then it does not effect another VM.
→ RollBack is easy.

* Disadvantage

→ SPOF → single point of failure (main machine failure) until & unless we have taken backup.

Q What is the diff b/w virtual machines & containers ?

LAB :- 2 EC2 [Elastic Compute Cloud] ~~Iaas~~

→ Creation of instance (Linux) & Details Access
→ Connection to instance & states of instance

* Instance type :-
T → Tiny
C → Compute Power
M → Memory (RAM)
D → datatype
EBS → elastic block storage
t2.micro
VPC → virtual private cloud.

- On demand Reserved instances
- Dedicated hosts, Spot Requests

23/1/25

Q What are the steps to launch ~~Linux~~ instance.

Ans Steps :- Access EC2 service on AWS management console.
Step 2 :- EC2 instance
Step 3 :- Launch instance
Step 4 :-
On Linux — SSH traffic are allowed
→ Machine create
→ Check machine details
→ Connect the machine
- if doesn't connect
→ check security group (forget port to open, ip addrs is restricated — , if key pair file is corrupt or unavailable

- Custom AMI
- Discrete Data

Page No.
Date : / /

* If you want to connect to windows instance using RDP
→ windows name / public ip / public DNS
→ User name
→ password of machine

2/2/25
LAB-3

Auto Scaling

Elasticity → (Horizontal scaling) { no. of servers increasing ↑

Scalability ↑ (vertical scaling) { RAM increasing [2GB - 8GB]

Autoscaling → increasing no. of servers according to user crowd.

Regions →

Availability zones

Data Center → Server / N/W , Power Supply , AC unit

```
sudo su -
yum install stress
stress -c no
-- --cpu 5/10
sudo shutdown now
```

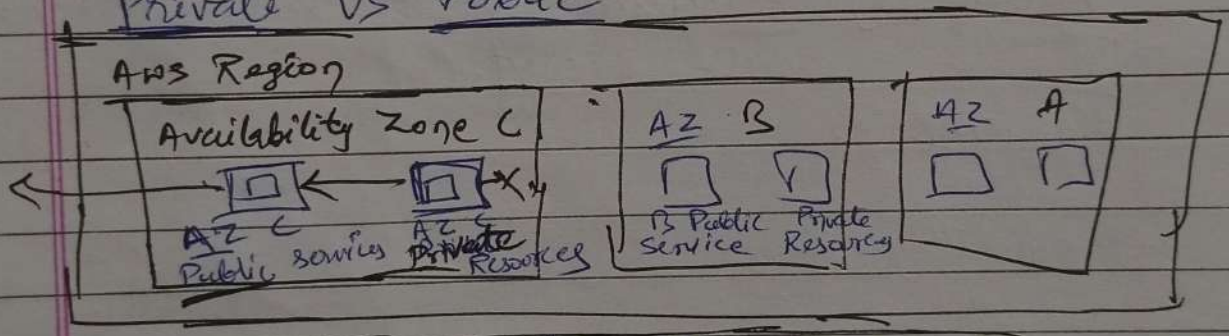
Dynamic scaling policy for Auto Scaling group & monitoring of EC2/AutoSc...

Amazon Web Services.

→ Amazon company
→ On demand cloud, Pay what u use or Reserve
→ Compute, Storage, Database, Media, ML, Analytics, Satellite Data, IOT & so much more

Compute	Storage & Access	Data transfer

AWS Global Infrastructure
Region → Physical location
Availability Zone → Data center
Regions are interconnected with AWS N/w.
Redundant power & Networking
Private Vs Public



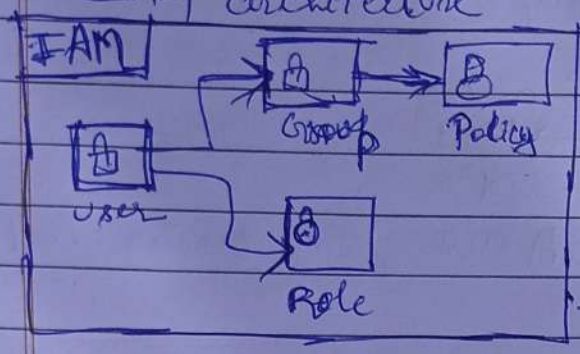
Customer Responsibility for Security IN Cloud	Platform, APP2, IAM, OS, N/w Firewall configuration
AWS Responsibility for Security 'OF' the cloud	Compute / Storage / DB / NW Hardware / AWS global infra. Regions / Avai. Zones / Edge Locations

↓
→ AWS is for Physical responsibility
→ User, Data, Encryption, EC2, Scaling. etc. is your responsibility.

Identity & Access Management

Page No.:
Date: / /

IAM Architecture



User, Group, Role & Policy.

→ Group is a way to organize user.
→ Policy define permission
→ User can get permission directly or via group.
→ Roles are for delegation & are assumed.
→ With assumed roles you can talk to other services.

→ Root User (signup AWS) has full permission
  → should be Avoided & enable MFA

→ Create new User
  → no permission by default
  → You can create around 5000 accounts (+more)
  → Use policies & prefer groups
  → Amazon Resource Name ARN & Regular Name
  → Username & Password OR Access keys (CLI)

→ Create Groups → Admin, Developer, Tester
  → Add policy to groups.
  → Add permission to group
  → User gets permission via group
  → Least priviledge Possible

→ IAM Roles.
  ~~Policies are JSON documents~~
  → Roles are assumed by users, applications & services.
  → Gain the permission of role only

→ IAM policies
  → Policies are JSON documents
  → Implicitly Denied
  → (1) Identity based & (2) Resource based policy
  → AWS support 6 types of policies
  (3) Permission Boundaries (5) Access Control List
  (4) Organization Service Ctrl Policy (6) Session Policy.

Page No.:
Date 29/1/25

LAB 1 → VM Creation (Hypervisor)

LAB 2 EC2 → Instances
Linux → Portal / Putty
Windows → RDP

LAB 3 → Custom AMI from running instance
nginx
Apache

LAB 4 → S3 bucket (server)
Simple storage service. (IAAS)
Container will contain files.

- item ownership
- making the obj public using ACL (access control list)
- disadv of using ACL method is whenever you load the new object in bucket you have to make it public manually.
- making the obj in the bucket public using bucket policy.
- Policies are the rules that are applied on the resources in amazon. These policies are statement written in json
- Policy generator is a tool for creating policies

31/1/25 # Diff b/w Storage Class based on following points

- Access Time
- I/O rate changes.
- diff in payment plans.
  → how many storage
  → how many requests for data access.
  → hardware for data storage

Versions
- custom images using
Page No.:
Date: / /

Standard
* I/A → Infrequent access.
* Standard is the default storage class
* Storage class bucket ki hoga ya object ki
→ It will of object (Storage class is applied at object level.)

5/2/24 LAB

VPC → virtual private cloud

10.0.0.0 - CA
172.16.0.0 - CB
192.168.0.0 - CC

NW bit | host bit

create VPC → 192.168.0.0/16

NW bit | host bit
192.168.1.0/24 → $2^8 - 2$

subnet
- subnet 1 : Public subnet 1 — 192.168.1.0/24
- " 2 : " " 2 — 192.168.2.0/24
- " 3 : Private subnet 1 — 192.168.3.0/24
- " 4 : " " 2 — 192.168.4.0/24

Private ip 10.0.0.0 → 1225

① VPC → ② subnet create → ③ create instance with VPC
④ create internet gateway → route table

→ Access instance with private ip from public instance
* ssh -i Pem key username@IP
-o pen
ppk

ESC
:wq
save script in vim

Laptop ↔ public linux ↔ public linux — ip print

NAT — N/w address translation
NAT Instance

1. if it doesn't connect instance → Ac² → networking → change source dest² → stop

10/3/25

Page No.:
Date: / /

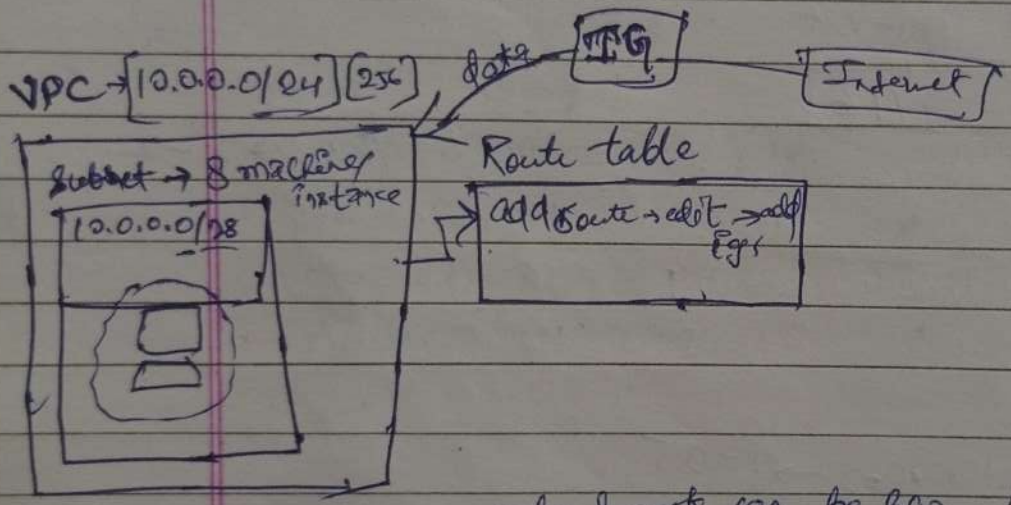
LAB-4 _ VPC

Scenario 1

TASK → ☐ Creating VPC

↳ Isolated N/W env for providing resources

- ☐ Subnet → creation
- ☐ Route table → creation / association
- ☐ Internet gateway
- ☐ EC2 instance launch (Public)
- ☐ Connection to instance.



subnet route can be b/w /16 to /28

10.0.0.0/24

10.0.0.0/28

0000 | 0000 → Subnet 1

(16 ← 0001 | 0000 → " 2

← 0010 | 0000 → " 3

32

Instance creation problem

→ Internet gateway should be attached

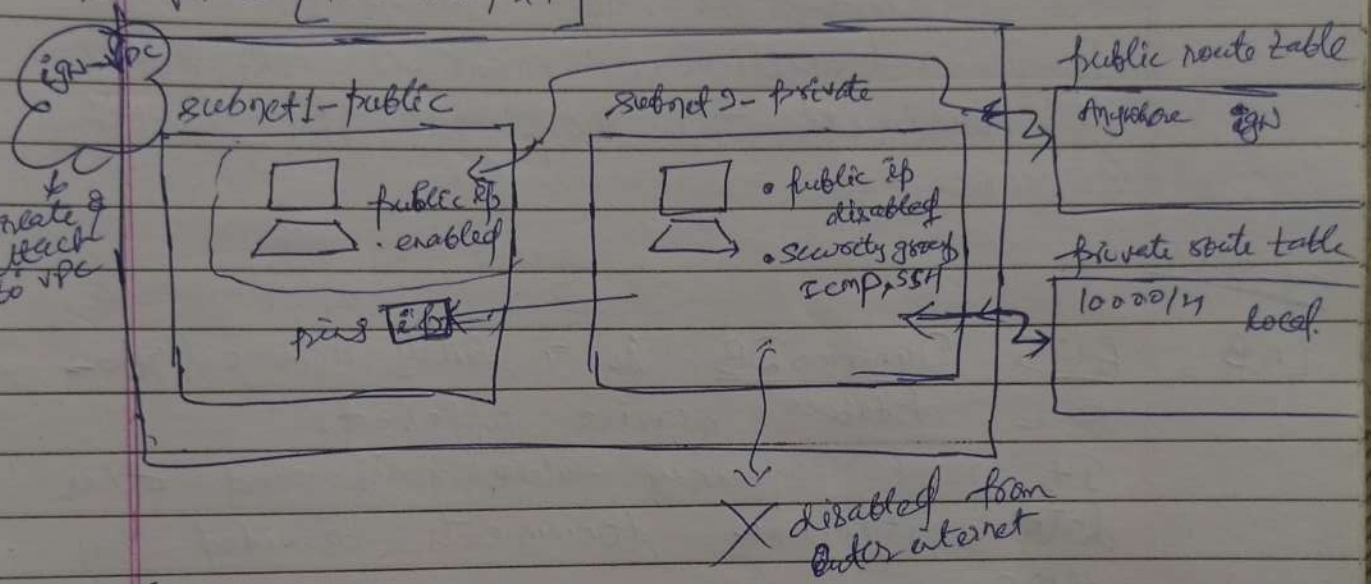
→ Route table ~~created~~ should contain the route for external communication (internet) — anywhere

→ subnet association with the route

→ auto assign public IP ~~the~~ option should be enabled.

→ In security group, SSH traffic (port 22) should be open.
→ For permanent public IP we get it by Elastic IP.

* VPC [10.0.0.0/24]



20/2/25

Elastic load balancing automatically distributes your traffic across multiple target such as EC2 instances, containers & IP addresses in one or more availability zones.

* There are 3 types of load balancers available.
  - Applic2 load balancer
  - N/w load balancer
  - Gateway load balancer

* Main advantages of using Load Balancer
  i. It balances the load on the appl2 server so that no one server should be overloaded
  ii. Client gets the single point of contact

It sends the req. to ELB and ELB redirects to the servers on the listening ports therefore if any of the server stops working or comes in the state of unhealthy then the client do not suffers.
ELB automatically monitors the state & redirect the req. to the healthy servers.

24-2-25

LAB AWS DynamoDB is a fully managed No-SQL database service offered.
It works on key value pair and other data structures documents provided by AWS.

Q What is the diff. b/w MongoDB & DynamoDB?

Q What is the diff b/w SQL & No-SQL DB?
relational ds (tabular) schema less

Main advantages of using DynamoDB.
- It is schema less therefore it can be used for any type of application.
- It is scalable. We can add as many table as we can.
- It is fully managed by AWS therefore we don't require any maintenance activity.
- It is very easy to use.
- Go

Physical Sch.
↓
Logical "
↓
View level "

Table
↓
Items
↓
Attributes

Components of DynamoDB

→ Attributes store the data
→ Item (document) is a set of attributes.
→ Table is the set of items. One item can be seen as a record.
→ The partition key in dynamoDB should be unique & not null.
→ Secondary Index is the attribute that is most often used to search the table. It can be used without partition key.
→ It gives the alternate access to the table
→ There is one more optional feature in dynamoDB that is known as dynamoDB stream. It keeps the track of data modification events in a table. Each event is represented by a stream record.

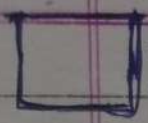
Ways to access DynamoDB

There are 3 ways to access dynamoDB.

1) Through Control Panel (GUI)
2) Amazon CLI
3) AWS SDKs

Create table → Create item → Add Attribute

Configuring the AWS CLI for working on CLI



→ AWS Portal

↳ AWS CLOUD SHELL

→ To access cloud shell from portal itself.
Cloudshell → shell interpretator (command line)

. sudo su -

. aws dynamodb list-tables

↳ (permission) Open the iam services ⇒ go to users ⇒ select that user & go to ⇒ security credentials ⇒ create access key if not available ⇒ (creating) click use case → next → description (optional) ⇒ generate (key generated) ⇒ download access key ⇒

. aws configure ↴ (to access dynamodb)

Access key id: - - - - - - - - - -

Access key → access key of
secret access key

secret access key: - - - - - - - -

aws region name: ap-south-01

quit output format [json] : json

6/3/25

DynamoDB through CLI → EC2 / powershell

→ Install AWS CLI

→ AWS CLI configuration

aws configure ↵

Access key ID:

Secret Access Key:

Default Region: us east-1

Default output format: json

aws cred doc

admin
admin
admin

Page No.:
Date: / /

• aws dynamodb list-tables

+ CLI CRUD ~~system~~ syntax (operations type)
- Put item
- get-item
- scan query
- delete item, table
- select item, table
- update item
- Create table

RDS → (Relational database service) & Aurora
→ RDS
→ Subnet groups → create DB subnet group

#/s/os

Cloud Watch → SaaS service
→ Amazon Cloud watch monitors & manages resources & applications running on AWS
→ It provides ~~help~~ health and performance matrix for AWS resources and applications.
→ Cloud watch is used to collect and track matrix data to measure the health and performance of aws resources. We can also create custom matrix and dashboards to visually display the data in graphical formats.
→ Cloud ~~watch~~ watch service can be used for creating alarms and can be integrated with other services like SNS & SQS for notifications.

notifications.

→ Cloud watch gives you system wide visibility into resource utilization, application health and operational health.

Creating an EC2 instance
CPU utilization threshold set

* IAM → Identity & Access Management (SAAS service)
* It provides access ctrl for identifiers
* It is a global service.
* Whenever we sign aws, it provides root account and it has all the priviledges.
* According to hirarchy of company it creates Account-
* Least priviledge principle
  - Permission Grand
  - IBAC → identity based access ctrl
  - RBAC → role based access control
  - ABAC → attribute based access control

* A user can be a part of more than 1 group.
* Explicit deny has highest priority.
* Roles are created for short term & is easy for switching users.

Intro to AWS Kinesis & Lambda Service

Page No.:
Date: 27/3/25

* Kinesis → real time data streaming, cloud-based service
→ data in bulk amount/real time data → use Kinesis

* AWS Lambda (serverless)
to use lambda service we will use SDK

Trigger source: API Gateway, S3, DynamoDB, CloudWatch etc.
func code: written in Python, Node.js, Java etc.
Execution Role: IAM permission for Lambda

Benefits:-
Scalability, cost-effective, ease of use.

* Create a role to work on lambda service for specific service in IAM.
* working service → S3, CloudWatch, Kinesis
create role → AWS service → services (select service)
→ role name → role created

Lambda f^2
create f^2 → f^2 name → select runtime →
choose role → f^2 data → created f^2

Create S3 bucket
bucket → properties → event notification centre →
create event notification →

create data stream → name → Data stream setting
→ enable encryption

Page No:
Date: 16/4/25

AWS Cloudformation

→ AWS Cloudformation is used for Creating & managing the Resources.

→ IaaS service

→ Infrastructure as a code.
↳ generating resource using code.

* Advantage (DevOps/Cloud)
  1 → Automate [Repitition task reduced]
  2 → Reduce time (replication work) → automate resource provisioning.
  3 → Github involvement → code is written in file called template and can be managed by github and versioning can be done too.
  4 → Order of resources in which it is to be written managed by cloudformation
  5 → Rollback → Rollbacks are automated

* Disadvantage
  - Adaptation of this technology is not begineer friendly.
  - Some situation, changes to template are too complex to handle.

Template Structure

Sections:
  AWS Template Format Version
  Description
  Parameters → dynamic values container while creating stack
  Resources (core)
  Outputs → gives value after generating resources

Written in YAML (preferred) or JSON

Page No.:
Date: / /

Example Template

→ Resources:

MyS3Bucket ⇒ (logical id)

Type: AWS::S3::Bucket

Properties:

BucketName: my-example-Bucket (unique)

Stack Management

- Stacks: Deployed instances of templates
- Stack Sets: Manage stacks across multiple accounts/regions
- Life cycle: Create → Update → Delete
- Rollback support on failure

Advanced Features.

Nested Stacks:

Drift Detection:

Change Sets:

CI/CD pipelines